

Evolutionary Trajectory Fine-Tuning (ETFT): Reverse-Engineering Algorithmic Progression and the Recursive Stage-Gate Calibration for SOTA+x Discovery

Lukas Benda

lukas.benda@boldpivot.cz

BoldPivot

April 2026

Abstract

Current Large Language Models (LLMs) suffer from architectural stagnation when faced with extreme resource constraints, primarily due to their training on static code snapshots. This paper presents the full technical framework for **Evolutionary Trajectory Fine-Tuning (ETFT)**[cite: 5]. By leveraging precisely engineered trajectories of functional algorithms from "Primordial" baselines to State-of-the-Art (SOTA) implementations [cite: 6, 7] we demonstrate how an LLM can learn the *vector of innovation* itself [cite: 18]. Crucially, we introduce a **Calibration Engine** that utilizes an "Evolutionary Replay" protocol. This establishes a hard algorithmic stage-gate, ensuring the model's hypothesis for next-generation algorithms (SOTA+1) is grounded in verifiable computer science principles rather than stochastic pattern matching.

1 The Morphological Constraint: Solving Granularity

A central focus of ETFT is calibrating the "size" of an evolutionary leap [cite: 85]. We solve this through a biological analogy of morphological shifts.

- **Macro-Evolutionary Jumps:** Transitions are not defined by syntactic edits, but by fundamental changes in complexity or representation, such as shifting from dense matrices to sparse ALBERT-style architectures [cite: 34, 43].
- **Operational Integrity:** Every step in the trajectory must be a fully functional, compilable program [cite: 39]. This prevents the model from learning "noisy" or broken code [cite: 26, 27].
- **Causal Meta-Data ($\mathcal{M}$):** Each transition is paired with quantitative impact data (e.g., "Freed 2.1 MB of code payload") and causal explanations [cite: 49, 52, 53].

2 The Calibration Engine: Evolutionary Replay

To satisfy the rigor required for autonomous research, the ETFT framework employs a **Replay Mechanism** to validate the model's readiness.

2.1 Formal Calibration Protocol

Before generating a SOTA+1 hypothesis, the model must "re-run" the history of the algorithm. Let $\mathcal{A}$ represent the set of compilable algorithms[cite: 29]. We define a fitness function $\mathcal{F}(a)$ inversely proportional to validation loss under a 16 MB constraint[cite: 30].

$$C = \frac{1}{n} \sum_{i=1}^n \text{Sim}(a_i^{\text{pred}}, a_i^{\text{true}}) \quad (1)$$

Where C is the **Confidence Level**. The system utilizes this as a hard algorithmic stage-gate:

1. **Input:** The model receives a_{i-1} and the target performance metrics from $\mathcal{M}_i$.
2. **Reconstruction:** The model must synthesize a_i that satisfies the ΔLoss and ΔMemory requirements[cite: 51, 72].
3. **Calibration Gate:** Only when C exceeds a predefined threshold is the model deemed "Ready for Action" to extrapolate toward a_{n+1} .

3 Recursive SOTA+x Discovery

The power of ETFT lies in its recursive potential. Once a model successfully hypothesizes a SOTA+1 architecture (a_{n+1}), it is verified on an $8 \times H100$ cluster[cite: 81].

- **Feedback Loop:** Verified a_{n+1} candidates are integrated back into the training trajectory[cite: 70].
- **Multi-Generational Innovation:** By recalibrating on the newly extended fossil record ($a_0 \rightarrow a_{n+1}$), the system prepares to hypothesize a_{n+2} and beyond[cite: 91].

4 Technical Summary for YC Review

Feature	Function	Verification Metric
Trajectory (T)	Ordered n-tuple of functional code [cite: 31]	$\mathcal{F}(a_{i-1}) < \mathcal{F}(a_i)$ [cite: 32]
Meta-Dataset (M)	Causal grounding of innovations [cite: 46]	Quantitative Resource Deltas [cite: 52]
Calibration	Evolutionary Replay validation	Reconstruction Confidence (C)
Stage-Gate	Unlock speculative SOTA+1 synthesis	Functional Parity with Historical SOTA

Table 1: The ETFT Readiness and Calibration Framework.

5 Conclusion

ETFT transitions LLMs from being passive coding assistants to being **Autonomous Computer Science Research Agents**[cite: 9, 91]. By treating algorithmic development as a discrete gradient ascent, we provide a path to independently discover the next generation of architectures for the world’s most constrained hardware environments.